

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

PROIECT DE DIPLOMĂ

Conducător științific: Șef lucr. Alexandru DOROBANȚIU
Îndrumător: Șef lucr. Alexandru DOROBANȚIU

Absolvent:
Răzvan-Bucur HOADREA
Specializarea CALCULATOARE

UNIVERSITATEA “LUCIAN BLAGA” DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE ȘI INGINERIE ELECTRICĂ

TITLUL TEMEI

Comparare algoritmi de predicție cu învățare automată

Conducător științific: Șef lucr. Alexandru DOROBANȚIU

Îndrumător: Șef lucr. Alexandru DOROBANȚIU

Absolvent:
Răzvan-Bucur HOADREA
Specializarea CALCULATOARE

UNIVERSITATEA "LUCIAN BLAGA" DIN SIBIU
FACULTATEA DE INGINERIE
DEPARTAMENTUL DE CALCULATOARE
ȘI INGINERIE ELECTRICĂ

APROBAT (data)

DIRECTOR DEPARTAMENT

PLAN TEMATIC
pentru proiectul de diplomă

Proiectul de diplomă dat studentului: Răzvan-Bucur HOADREA

- 1. Tema proiectului:** Comparare algoritmi de predicție cu învățare automată
- 2. Termenul de predare a proiectului:** Iunie 2026
- 3. Elemente inițiale pentru proiect:** Învățare automată, Benchmark competiție sportivă
- 4. Nota explicativă (enumerarea problemelor care vor fi rezolvate):**
 - Implementarea unui model de învățare automată
 - Aplicare de predicții pe baza seturilor de date
 - Efectuarea unui studiu comparativ între mai mulți algoritmi de învățare automată
- 5. Enumerarea materialului grafic (cu indicarea precisă a desenelor obligatorii):**
 - Schema arhitecturii algoritmilor de învățare automată
 - Graficul comparativ al performanței algoritmilor
- 6. Consultații pentru proiect (săptămânal):** MIERCURI 14:00–15:00
- 7. Data eliberării temei:** OCTOMBRIE 2025.

Tema a fost primită pentru îndeplinire:
Data 31.10.2025

CONDUCĂTOR,
(grad didactic, prenume, nume)

Semnătură student

Șef lucr. Alexandru DOROBANȚIU

(semnătura)

Cuprins

1	Introducere	3
1.1	Prezentarea temei	3
1.2	Motivație	4
1.3	Scop și obiective	4
1.4	Structura lucrării	4
2	Considerații teoretice	5
2.1	Învățarea automată și clasificarea supervizată	5
2.2	Problema dezechilibrului de clase	5
2.3	Regresia logistică	6
2.4	Arbori de decizie	6
2.5	Pădurea aleatoare (Random Forest)	7
2.6	Ingineria caracteristicilor și scurgerea de informație	7
2.7	Validarea și estimarea performanței	8
2.8	Metrici de evaluare	8
2.9	Lucrări conexe	8
3	Rezolvarea temei de proiect	10
3.1	Arhitectura sistemului	10
3.2	Sursa de date	11

3.3	Construirea setului de date	11
3.4	Ingineria caracteristicilor cauzale	11
3.5	Implementarea regresiei logistice	12
3.6	Implementarea arborelui de decizie	12
3.7	Implementarea pădurii aleatoare	13
3.8	Predicția podiumului și agregarea în campionat	13
3.9	Validarea temporală pe ani	13
3.10	Metrici și instrumentația experimentală	14
4	Rezultate	15
4.1	Configurația experimentală	15
4.2	Performanța comparativă	15
4.3	Capacitatea de discriminare: ROC și Precision-Recall	16
4.4	Analiza hiperparametrilor	17
4.5	Costul de calcul	17
4.6	Diagnostic de antrenare	19
4.7	Sensibilitatea la pragul de decizie	19
4.8	Discuție: acuratețea în prezența dezechilibrului	20
4.9	Studiu de caz: sezonul 2025	20
5	Concluzii și dezvoltări ulterioare	22
5.1	Concluzii	22
5.2	Dezvoltări ulterioare	23
	Bibliografie	24

Capitolul 1

Introducere

1.1 Prezentarea temei

Predicția rezultatelor sportive este un domeniu de aplicație clasic pentru învățarea automată, deoarece combină date structurate, abundente și actualizate periodic cu o problemă de decizie clar definită. Formula 1 oferă un cadru deosebit de potrivit: fiecare sezon generează zeci de curse, fiecare cursă produce rezultate pentru aproximativ douăzeci de piloți, iar regulile competiției sunt stabile pe perioade de mai mulți ani. Spre deosebire de sporturile de echipă, în care rezultatul depinde de interacțiuni complexe între mulți jucători, în Formula 1 performanța individuală a pilotului și a echipei poate fi urmărită numeric de la o cursă la alta.

Lucrarea de față abordează problema predicției *podiumului* unei curse de Formula 1, adică a identificării celor trei piloți care vor termina pe primele trei locuri (P1, P2 și P3). Problema este formulată ca o sarcină de clasificare binară: pentru fiecare pereche (cursă, pilot) se estimează probabilitatea ca pilotul respectiv să termine pe podium, după care, pentru fiecare cursă, piloții sunt ordonați descrescător după această probabilitate, iar primii trei formează podiumul prezis. Pe lângă predicția per cursă, probabilitățile sunt agregate într-un clasament de campionat estimat la finalul sezonului.

Elementul central al lucrării nu este atât aplicația în sine, cât **studiul comparativ** al trei algoritmi de învățare automată implementați integral, fără utilizarea unor biblioteci de învățare automată gata făcute: regresia logistică, arborele de decizie și pădurea aleatoare (*Random Forest*). Implementarea de la zero, peste biblioteca de calcul numeric NumPy [1], urmărește înțelegerea în profunzime a mecanismelor fiecărui algoritm și controlul complet asupra procesului de antrenare și evaluare.

1.2 Motivație

Alegerea temei este motivată de trei aspecte. În primul rând, implementarea de la zero a algoritmilor expune detaliile pe care bibliotecile consacrate, precum scikit-learn [2], le ascund: coborârea pe gradient, criteriul de divizare bazat pe entropie sau mecanismul de *bagging*. În al doilea rând, datele din Formula 1 ridică o provocare metodologică reală — *scurgerea de informație* (eng. *data leakage*) —, deoarece orice caracteristică folosită pentru a prezice o cursă trebuie calculată exclusiv din rezultate anterioare acelei curse. În al treilea rând, distribuția claselor este puternic dezechilibrată (doar aproximativ 15% dintre piloți termină pe podium), ceea ce face ca metrica de acuratețe globală să fie înșelătoare [3], [4] și impune folosirea unor metrice adecvate.

1.3 Scop și obiective

Scopul lucrării este realizarea unui studiu comparativ riguros între trei algoritmi de învățare automată aplicați aceleiași probleme de predicție, în condiții identice de antrenare și evaluare. Obiectivele concrete, în acord cu planul tematic al proiectului, sunt următoarele:

1. **Implementarea de la zero** a trei modele de învățare automată (regresie logistică, arbore de decizie și pădure aleatoare), folosind doar NumPy pentru algebra vectorizată.
2. **Aplicarea predicțiilor** pe un set de date real, alcătuit din rezultatele curselor de Formula 1 din sezoanele 2018–2025, cu o etapă de inginerie a caracteristicilor strict cauzală, fără scurgere de informație.
3. **Efectuarea unui studiu comparativ** între cei trei algoritmi, atât din punctul de vedere al calității predicțiilor (acuratețe, F1, rată de nimerire a podiumului, arie sub curbele ROC și Precision-Recall), cât și al costului de calcul (timpul de antrenare).
4. **Analiza hiperparametrilor** fiecărui model, prin curbe de validare și căutare pe grilă, pentru a justifica valorile alese.

1.4 Structura lucrării

Lucrarea este organizată în cinci capitole. Capitolul 2 prezintă fundamentele teoretice ale învățării automate și ale celor trei algoritmi studiați, împreună cu metodele de validare și metricile de evaluare. Capitolul 3 descrie soluția dezvoltată: arhitectura sistemului, sursa de date, ingineria caracteristicilor cauzale, implementarea algoritmilor și a procedurii de validare. Capitolul 4 prezintă și interpretează rezultatele experimentale. Capitolul 5 sintetizează concluziile și propune direcții de dezvoltare ulterioară.

Capitolul 2

Considerații teoretice

2.1 Învățarea automată și clasificarea supervizată

Învățarea automată studiază algoritmi care îmbunătățesc performanța la o sarcină pe baza experienței, exprimată sub forma unor date [5]. În cadrul *învățării supervizate*, algoritmul primește un set de exemple de antrenament $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$, unde $\mathbf{x}_i \in \mathbb{R}^d$ este un vector de caracteristici, iar y_i este eticheta asociată. Scopul este învățarea unei funcții $f : \mathbb{R}^d \rightarrow \mathcal{Y}$ care generalizează bine pe date nevăzute.

Atunci când eticheta ia două valori, $\mathcal{Y} = \{0, 1\}$, problema se numește *clasificare binară*. În lucrarea de față, eticheta $y = 1$ semnifică faptul că pilotul termină pe podium (locul ≤ 3), iar $y = 0$ în caz contrar. Multe modele de clasificare nu produc direct eticheta, ci o *probabilitate* $\hat{p} = P(y = 1 \mid \mathbf{x})$, pe care o transformă într-o etichetă printr-un prag de decizie (uzual 0,5). Această probabilitate este esențială în soluția propusă, fiindcă servește drept scor de ordonare a piloților în cadrul fiecărei curse.

2.2 Problema dezechilibrului de clase

Un set de date este dezechilibrat atunci când clasele nu sunt reprezentate în proporții apropiate. În cazul predicției podiumului, doar trei din aproximativ douăzeci de piloți ajung pe podium în fiecare cursă, deci clasa pozitivă reprezintă circa 15% din exemple. În astfel de situații, un clasificator trivial care prezice mereu clasa majoritară (“nu termină pe podium”) obține o acuratețe ridicată (aproximativ 85%) fără a avea nicio valoare practică [3]. De aceea, evaluarea trebuie completată cu metrici care surprind performanța pe clasa minoritară, precum scorul F1 și aria sub curba Precision-Recall [4]. O tehnică suplimentară de atenuare a dezechilibrului este ponderarea exemplilor pe clase în funcția de cost, aplicată în această lucrare la regresia logistică.

2.3 Regresia logistică

Regresia logistică modelează probabilitatea clasei pozitive printr-o transformare neliniară a unei combinații liniare de caracteristici [6]. Fie $\mathbf{w}$ vectorul de ponderi și b termenul liber; modelul calculează scorul $z = \mathbf{w}^\top \mathbf{x} + b$ și îl transformă prin funcția *sigmoid*:

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \quad \hat{p} = \sigma(\mathbf{w}^\top \mathbf{x} + b). \quad (2.1)$$

Antrenarea minimizează funcția de cost de tip *entropie încrucișată* (eng. *cross-entropy*), la care se adaugă un termen de regularizare L_2 ce penalizează ponderile mari și combate supraînvățarea:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{N} \sum_{i=1}^N \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \frac{\lambda}{2} \|\mathbf{w}\|_2^2. \quad (2.2)$$

Minimizarea se realizează prin *coborâre pe gradient* (eng. *gradient descent*): ponderile sunt actualizate iterativ în sensul opus gradientului, cu un pas de învățare η . Gradientul funcției de cost față de ponderi are forma compactă:

$$\nabla_{\mathbf{w}} \mathcal{L} = \frac{1}{N} \mathbf{X}^\top (\hat{\mathbf{p}} - \mathbf{y}) + \lambda \mathbf{w}. \quad (2.3)$$

Pentru a contracara dezechilibrul de clase (Secțiunea 2.2), fiecare exemplu poate primi o pondere invers proporțională cu frecvența clasei sale, astfel încât erorile pe clasa minoritară (podium) să contribuie semnificativ la funcția de cost.

2.4 Arbori de decizie

Un arbore de decizie partiționează recursiv spațiul caracteristicilor prin teste de forma “caracteristica j este $\leq$ pragul t ?” [7]. Fiecare nod intern aplică un astfel de test, iar fiecare frunză stochează o predicție — în cazul de față, fracția de exemple pozitive din frunză, interpretată drept probabilitate de podium.

Criteriul de alegere a divizărilor folosit în această lucrare este *câștigul de informație* (eng. *information gain*), bazat pe *entropie*. Pentru un nod cu proporția p de exemple pozitive, entropia binară este:

$$H(p) = -p \log_2 p - (1 - p) \log_2 (1 - p), \quad (2.4)$$

cu convenția $0 \log_2 0 = 0$. Câștigul de informație al unei divizări care împarte un nod părinte în doi copii (stânga și dreapta) este reducerea de entropie:

$$IG = H(\text{părinte}) - \frac{n_{st}}{n} H(\text{stânga}) - \frac{n_{dr}}{n} H(\text{dreapta}). \quad (2.5)$$

Algoritmul alege, la fiecare nod, perechea (caracteristică, prag) care maximizează câștigul de informație. Creșterea arborelui se oprește la atingerea unor criterii precum adâncimea maximă (`max_depth`), numărul minim de exemple necesare pentru divizare (`min_samples_split`) sau numărul minim de exemple într-o frunză (`min_samples_leaf`). Aceste criterii controlează compromisul dintre subînvățare și supraînvățare.

2.5 Pădurea aleatoare (Random Forest)

Pădurea aleatoare este o metodă de tip ansamblu care combină predicțiile mai multor arbori de decizie pentru a reduce varianța și a îmbunătăți generalizarea [8]. Doi factori de aleatoriu decorează arborii:

- **Bagging** (eng. *bootstrap aggregating*): fiecare arbore este antrenat pe un eșantion bootstrap, obținut prin extragerea cu revenire a n exemple din setul de antrenament.
- **Subset aleator de caracteristici**: la fiecare divizare se consideră doar un subset aleator de caracteristici (uzual $\sqrt{d}$), nu toate cele d disponibile.

Predicția finală este media probabilităților produse de toți arborii. Un avantaj important al metodei este estimarea *out-of-bag* (OOB): pentru fiecare exemplu, predicția se calculează folosind doar arborii care nu l-au inclus în eșantionul bootstrap, oferind o estimare a erorii de generalizare fără un set de validare separat [8].

2.6 Ingineria caracteristicilor și scurgerea de informație

Calitatea unui model de învățare automată depinde decisiv de caracteristicile de intrare [9]. În problemele cu o componentă temporală, precum predicția unei curse viitoare, apare riscul de *scurgere de informație*: utilizarea, la antrenare, a unor informații care nu ar fi disponibile în momentul predicției reale. De exemplu, folosirea numărului total de puncte acumulate de un pilot într-un sezon ar include rezultatul cursei pe care încercăm să o prezicem.

Pentru a evita acest fenomen, toate caracteristicile trebuie să fie *strict cauzale*, adică să folosească exclusiv informații anterioare cursei prezise. Strategia adoptată constă în procesarea curselor în ordine cronologică și menținerea unei “stări” istorice, din care se calculează caracteristicile înainte de a fi actualizată cu rezultatul cursei curente.

2.7 Validarea și estimarea performanței

Estimarea corectă a performanței necesită evaluarea modelului pe date care nu au fost folosite la antrenare [10]. Validarea încrucișată clasică (*k-fold*) presupune însă că exemplele sunt independente și interschimbabile, ceea ce nu este valabil pentru date temporale: nu putem antrena pe curse viitoare pentru a prezice curse trecute. De aceea, în această lucrare se folosește o *validare temporală pe ani* (eng. *rolling cross-year*): pentru fiecare sezon de test S , modelul este antrenat pe toate sezoanele anterioare ($< S$) și evaluat pe sezonul S . Astfel se prezice mereu “viitorul” cu modele antrenate doar pe “trecut”, eliminând scurgerea de informație temporală.

2.8 Metrici de evaluare

Pentru clasificarea binară, plecând de la numărul de adevărat pozitive (TP), fals pozitive (FP), adevărat negative (TN) și fals negative (FN), se definesc:

$$\text{Acuratețe} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{Precizie} = \frac{TP}{TP + FP}, \quad (2.6)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2 \cdot \text{Precizie} \cdot \text{Recall}}{\text{Precizie} + \text{Recall}}. \quad (2.7)$$

Scorul F_1 este media armonică dintre precizie și recall și este preferat acurateței în prezența dezechilibrului de clase. Curba ROC reprezintă rata adevărat pozitivelor în funcție de rata fals pozitivelor, la variația pragului de decizie; aria sub această curbă (AUC) este o măsură agregată a capacității de discriminare [11]. Curba Precision-Recall și aria sub ea (*average precision*, AP) sunt mai informative decât ROC atunci când clasa pozitivă este rară [4]. În fine, *curba de calibrare* (eng. *reliability diagram*) compară probabilitatea medie prezisă cu frecvența reală observată, indicând cât de bine pot fi interpretate probabilitățile ca atare [12].

Pe lângă metricile de clasificare, soluția definește metrici specifice problemei, calculate la nivel de cursă și apoi mediate: *rata de nimerire a podiumului* (câți dintre cei trei piloți preziși se regăsesc în podiumul real), acuratețea predicției câștigătorului și suprapunerea cu primii trei din clasamentul de campionat.

2.9 Lucrări conexe

Aplicarea învățării automate la predicția rezultatelor sportive a fost studiată pe larg. Bunker și Thabtah [13] propun un cadru general pentru predicția rezultatelor sportive și subliniază importanța ingineriei caracteristicilor și a evaluării temporale. Metodele de tip ansamblu, în special pădurile aleatoare, sunt frecvent raportate ca fiind robuste și competitive pe date tabulare [6], [8]. Lucrarea de față se

diferențiază prin implementarea integrală a algoritmilor și prin accentul pus pe analiza comparativă a hiperparametrilor și a costului de calcul.

Capitolul 3

Rezolvarea temei de proiect

3.1 Arhitectura sistemului

Sistemul este organizat ca o succesiune de etape (eng. *pipeline*), de la datele brute până la predicții și evaluare. Figura 3.1 prezintă schema de arhitectură: datele sunt încărcate din memoria cache locală, transformate într-un tabel de rezultate, apoi îmbogățite cu caracteristici cauzale; pe acest set se antrenează cele trei modele, ale căror probabilități sunt folosite atât pentru predicția podiumului per cursă, cât și pentru agregarea în clasament și pentru modulul de evaluare și vizualizare.

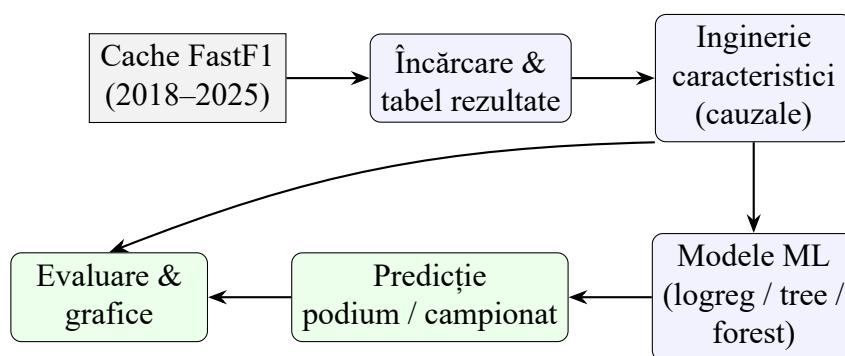


Figura 3.1 Arhitectura sistemului de predicție: de la datele brute la evaluare.

Implementarea este realizată în limbajul Python, organizată în pachetul `f1pred`. Bibliotecile externe sunt folosite strict pentru sarcini auxiliare: NumPy [1] pentru algebra vectorizată, pandas [14] pentru manipularea datelor tabelare, matplotlib [15] pentru grafice și FastF1 [16] pentru încărcarea datelor. Niciuna nu este o bibliotecă de învățare automată; toți algoritmi, metricile și procedura de antrenare sunt implementați manual.

3.2 Sursa de date

Datele provin din interfața de programare FastF1 [16], care oferă acces la rezultatele oficiale și la informațiile de cronometraj din Formula 1. Au fost folosite opt sezoane (2018–2025), însumând aproximativ 3500 de rezultate (perechi cursă–pilot). Pentru reproductibilitate și pentru a elimina dependența de conexiunea la internet, datele sunt citite dintr-o memorie cache locală, în mod strict offline.

3.3 Construirea setului de date

Prima etapă transformă datele brute într-un tabel “lung”, cu o linie pentru fiecare pereche (sezon, rundă, pilot), conținând poziția pe grilă, poziția în calificări, poziția finală, punctele, echipa și condițiile meteo. Tabelul este sortat cronologic — pas esențial pentru calculul corect al caracteristicilor rulante — și salvat în format Parquet pentru reutilizare rapidă.

3.4 Ingineria caracteristicilor cauzale

Pe baza considerentelor din Secțiunea 2.6, au fost construite 30 de caracteristici numerice, toate strict cauzale. Acestea se grupează în următoarele categorii, rezumate în Tabelul 3.1: poziția pe grilă și în calificări (cunoscute înainte de cursă), forma rulantă a pilotului (medii pe ultimele cinci curse), statistici cumulate la nivel de sezon, forța echipei, istoricul pe circuitul respectiv și contextul cursei (experiență, condiții meteo).

Tabelul 3.1 Categoriile de caracteristici folosite de modele.

Categorie	Exemple de caracteristici	Nr.
Grilă / calificări	poziție grilă, poziție calificări, pole	5
Formă rulantă pilot	medie sosire, medie puncte, rată podium, rată abandon	6
Sezon-to-date pilot	puncte, poziție, podiumuri, victorii, curse	5
Forța echipei	formă echipă, puncte echipă, poziție echipă	4
Istoric circuit	medie sosire pilot / echipă, rată podium pe circuit	3
Context	experiență, rundă, weekend cu sprint, vreme	7
Total		30

Principiul cauzalității este implementat prin actualizarea stării istorice *după* calcularea caracteristicilor cursei curente. Listarea 3.1 ilustrează această ordine pentru caracteristica “puncte acumulate în sezon”: valoarea atribuită cursei curente reflectă doar cursele anterioare.

```
1 # Pas 1: caracteristicile cursei R folosesc DOAR starea de pana acum.
2 row["season_points"] = state_points[driver]      # puncte inainte de cursa R
3
```

```

4 # Pas 2: abia DUPA ce s-au calculat caracteristicile,
5 #       starea este actualizata cu rezultatul cursei R.
6 state_points[driver] += points_in_race_R

```

Listarea 3.1: Actualizarea stării istorice dupa calcularea caracteristicilor (principiul anti-leakage).

3.5 Implementarea regresiei logistice

Regresia logistică (Secțiunea 2.3) a fost implementată prin coborâre pe gradient în varianta *batch*, cu regularizare L_2 și ponderare pe clase. Caracteristicile sunt standardizate intern (medie zero, abatere standard unitară), parametrii standardizării fiind învățați exclusiv pe datele de antrenament. Listarea 3.2 prezintă bucla de antrenare, care implementează ecuațiile (2.2) și (2.3) și înregistrează istoricul funcției de cost pentru analiza convergenței.

```

1 for _ in range(self.n_iters):
2     z = X @ self.w + self.b
3     p = _sigmoid(z)
4     # Pierdere (cross-entropy ponderata + L2) la pasul curent.
5     ce = -(sample_w * (y*np.log(p+eps) + (1-y)*np.log(1-p+eps))).sum() /
        sw_sum
6     self.loss_history_.append(float(ce + 0.5*self.l2 * float(self.w @
        self.w)))
7
8     err = (p - y) * sample_w                # gradientul cross-entropy,
        ponderat
9     grad_w = X.T @ err / sw_sum + self.l2 * self.w
10    grad_b = err.sum() / sw_sum
11    self.w -= self.lr * grad_w
12    self.b -= self.lr * grad_b

```

Listarea 3.2: Bucla de coborare pe gradient a regresiei logistice.

3.6 Implementarea arborelui de decizie

Arborele de decizie (Secțiunea 2.4) folosește criteriul entropie plus câștig de informație. Pentru eficiență, căutarea celui mai bun prag pe o caracteristică este vectorizată: valorile sunt sortate, iar prin sume cumulative ale etichetelor se obține instantaneu numărul de pozitive din partea stângă pentru fiecare prag posibil, deci entropiile copiilor pentru toate pragurile candidate simultan. Listarea 3.3 prezintă nucleul acestui calcul, corespunzând ecuațiilor (2.4) și (2.5).

```

1 cum_pos      = np.cumsum(y_sorted)                # pozitive cumulate in stanga
2 left_total   = np.arange(1, n)                    # numar exemple in stanga
3 left_pos     = cum_pos[:-1]
4 right_total  = n - left_total
5 right_pos    = total_pos - left_pos
6

```

```

7 h_left = _binary_entropy(left_pos / left_total)
8 h_right = _binary_entropy(right_pos / right_total)
9 child_entropy = (left_total/n)*h_left + (right_total/n)*h_right
10 gain = parent_entropy - child_entropy      # castigul de informatie

```

Listarea 3.3: Calculul vectorizat al castigului de informatie pentru toate pragurile unei caracteristici.

3.7 Implementarea pădurii aleatoare

Pădurea aleatoare (Secțiunea 2.5) este construită peste arborele de decizie propriu. Fiecare arbore primește un eșantion bootstrap al rândurilor și un generator de numere aleatoare propriu, ceea ce asigură reproductibilitatea și, în același timp, decorelarea arborilor. Listarea 3.4 prezintă bucla de antrenare a ansamblului.

```

1 for _ in range(self.n_estimators):
2     idx = rng.integers(0, n, size=n)      # esantion bootstrap (cu revenire)
3     tree = DecisionTree(
4         max_depth=self.max_depth,
5         min_samples_leaf=self.min_samples_leaf,
6         max_features=self.max_features,    # subset aleator de caracteristici
7         rng=np.random.default_rng(rng.integers(0, 2**32 - 1)),
8     )
9     tree.fit(X[idx], y[idx])
10    self.trees.append(tree)

```

Listarea 3.4: Antrenarea pe baza de bagging a padurii aleatoare.

3.8 Predicția podiumului și agregarea în campionat

Deoarece un clasificator binar aplicat independent fiecărui pilot nu garantează exact trei predicții pozitive pe cursă, podiumul este determinat prin *ordonare*: pentru fiecare cursă se calculează probabilitatea de podium a tuturor piloților, aceștia sunt sortați descrescător, iar primii trei formează podiumul prezis (probabilitatea cea mai mare corespunde locului P1). Egalitățile de probabilitate se departajează după poziția pe grilă. Pentru clasamentul de campionat, probabilitățile sunt convertite în puncte estimate pe cursă, apoi însumate pe sezon.

3.9 Validarea temporală pe ani

Procedura de validare implementează schema descrisă în Secțiunea 2.7. Listarea 3.5 prezintă bucla principală: pentru fiecare sezon de test, modelul este reantrenat de la zero pe sezoanele anterioare și evaluat pe sezonul curent.

```

1 for test_season in seasons[1:]:           # primul sezon nu are trecut
2     train = feats[feats["season"] < test_season]
3     test = feats[feats["season"] == test_season]
4     Xtr, ytr, _ = feature_matrix(train)
5     Xte, yte, _ = feature_matrix(test)
6
7     model = build_model(model_key, **overrides)
8     model.fit(Xtr, ytr)                   # antrenare doar pe trecut
9     proba = model.predict_proba(Xte)      # predictie pe sezonul de test

```

Listarea 3.5: Bucla de validare temporală pe ani (rolling cross-year).

3.10 Metrici și instrumentația experimentală

Toate metricile din Secțiunea 2.8 sunt implementate manual, inclusiv curbele ROC, Precision-Recall și de calibrare, precum și aria sub curbe (prin regula trapezelor). Pe lângă evaluarea de bază, sistemul include un modul de experimente care realizează: variația hiperparametrilor pe câte o dimensiune (curbe de validare), căutarea pe grilă 2D, măsurarea timpului de antrenare, analiza convergenței și a erorii out-of-bag, curba de învățare (în funcție de volumul de date) și sensibilitatea la pragul de decizie. Întreaga funcționalitate este expusă printr-o interfață în linie de comandă.

Capitolul 4

Rezultate

4.1 Configurația experimentală

Experimentele folosesc setul de date descris în Capitolul 3, cu validare temporală pe sezoanele 2019–2025 (fiecare sezon prezis cu un model antrenat pe sezoanele anterioare). Hiperparametrii de bază sunt: pentru regresia logistică, pas de învățare 0,1, 3000 de iterații și regularizare L_2 cu $\lambda = 0,01$; pentru arbore, adâncime maximă 8; pentru pădure, 60 de arbori și adâncime maximă 12, cu $\sqrt{d}$ caracteristici per divizare.

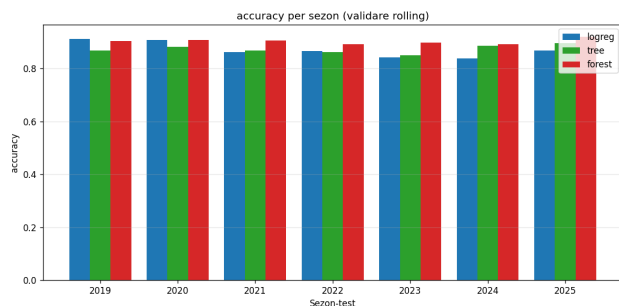
4.2 Performanța comparativă

Tabelul 4.1 prezintă performanța medie a celor trei modele pe sezoanele de test. Pădurea aleatoare obține cea mai bună acuratețe (0,904) și cea mai bună rată de nimerire a podiumului (0,691), în timp ce regresia logistică conduce la cel mai bun scor pentru predicția câștigătorului (0,550) și pentru suprapunerea cu primii trei din campionat (0,905). Arborele de decizie simplu este, în mod constant, modelul cel mai slab, ceea ce confirmă avantajul metodelor de ansamblu și al regularizării.

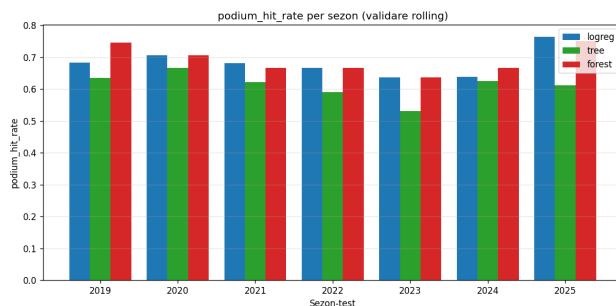
Tabelul 4.1 Performanța medie a modelelor pe sezoanele de test (validare temporală).

Model	Acuratețe	F1	Rată podium	Câștigător	Top-3 campionat
Regresie logistică	0,870	0,667	0,682	0,550	0,905
Arbore de decizie	0,874	0,580	0,611	0,445	0,857
Random Forest	0,904	0,664	0,691	0,466	0,810

Figura 4.1 prezintă, comparativ, acuratețea și rata de nimerire a podiumului defalcate pe sezoane. Se observă că rata de nimerire a podiumului (Figura 4.1b) este în mod sistematic mai mică decât acuratețea (Figura 4.1a), aspect discutat în Secțiunea 4.8.



(a) Acuratețe



(b) Rată de nimerire a podiumului

Figura 4.1 Performanța pe sezoane: acuratețe (a) și rată de nimerire a podiumului (b).

4.3 Capacitatea de discriminare: ROC și Precision-Recall

Figura 4.2 prezintă curbele ROC ale celor trei modele. Pădurea aleatoare și regresia logistică obțin valori AUC foarte apropiate (0,936, respectiv 0,934), net superioare arborelui simplu (0,834). Tabelul 4.2 rezumă valorile AUC și AP (aria sub curba Precision-Recall). Conform Secțiunii 2.2, curba Precision-Recall este mai informativă în prezența dezechilibrului de clase; aici regresia logistică obține cel mai bun AP (0,711).

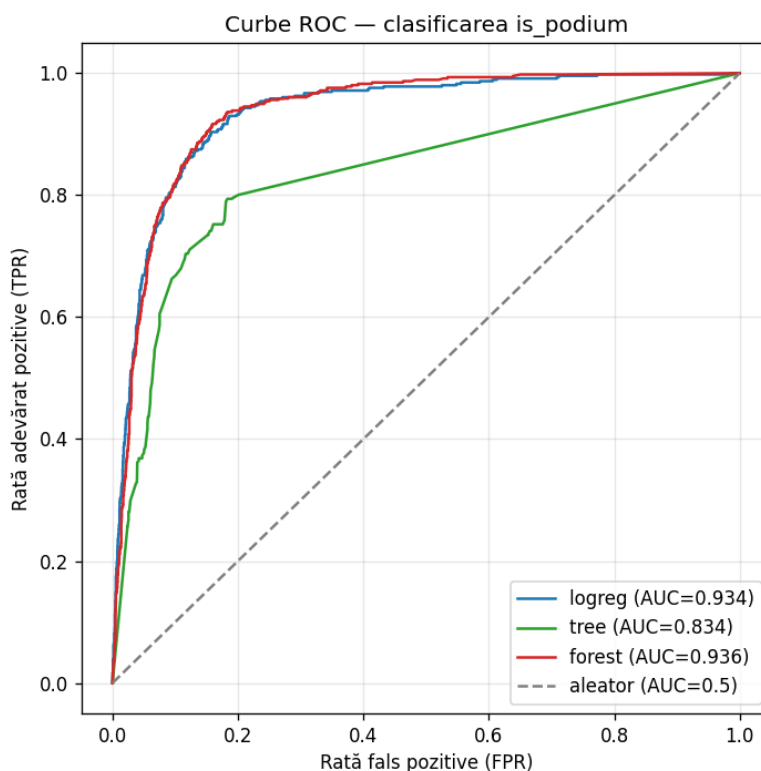


Figura 4.2 Curbele ROC pentru cele trei modele și valorile AUC asociate.

Tabelul 4.2 Aria sub curbele ROC (AUC) și Precision-Recall (AP).

Model	AUC	AP
Regresie logistică	0,934	0,711
Arbore de decizie	0,834	0,502
Random Forest	0,936	0,687

4.4 Analiza hiperparametrilor

Curbele de validare ilustrează efectul fiecărui hiperparametru asupra performanței și asupra timpului de antrenare. Figura 4.3 arată că, pentru pădurea aleatoare, creșterea numărului de arbori îmbunătățește marginal performanța, dar crește aproape liniar timpul de antrenare — un compromis tipic între calitate și cost. Căutarea pe grilă 2D (Figura 4.4) confirmă că arborele de decizie atinge cele mai bune valori pentru adâncimi moderate și un număr minim de exemple pe frunză suficient de mare pentru a evita supraînvățarea.

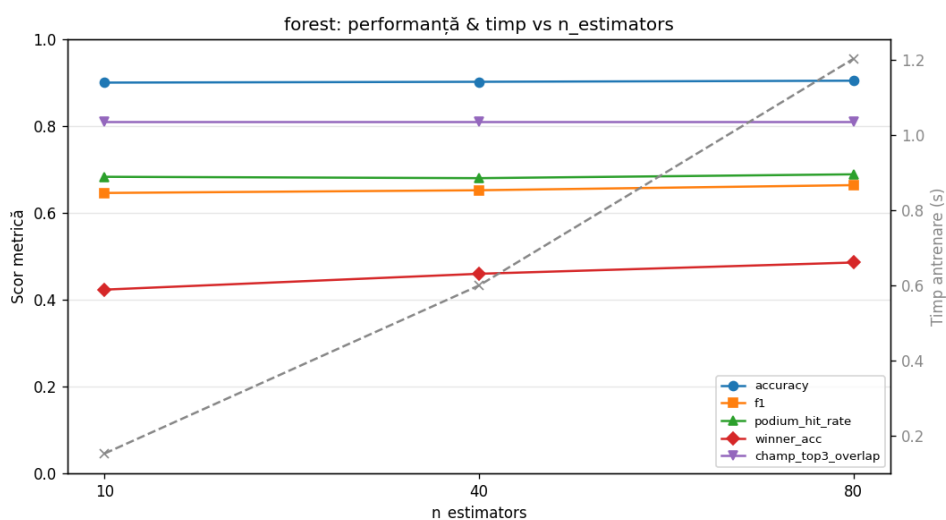


Figura 4.3 Curbă de validare: performanța și timpul de antrenare ale pădurii aleatoare în funcție de numărul de arbori.

4.5 Costul de calcul

Timpul de antrenare, măsurat pe setul complet de antrenament, este prezentat în Figura 4.5. Pădurea aleatoare este de aproximativ douăzeci de ori mai lentă decât regresia logistică (1,67 s față de 0,29 s), iar arborele simplu este cel mai rapid (0,09 s). Coroborat cu Secțiunea 4.2, acest rezultat evidențiază un compromis relevant pentru practică: regresia logistică oferă o capacitate de discriminare practic egală cu a pădurii (Tabelul 4.2), la o fracțiune din costul de calcul.

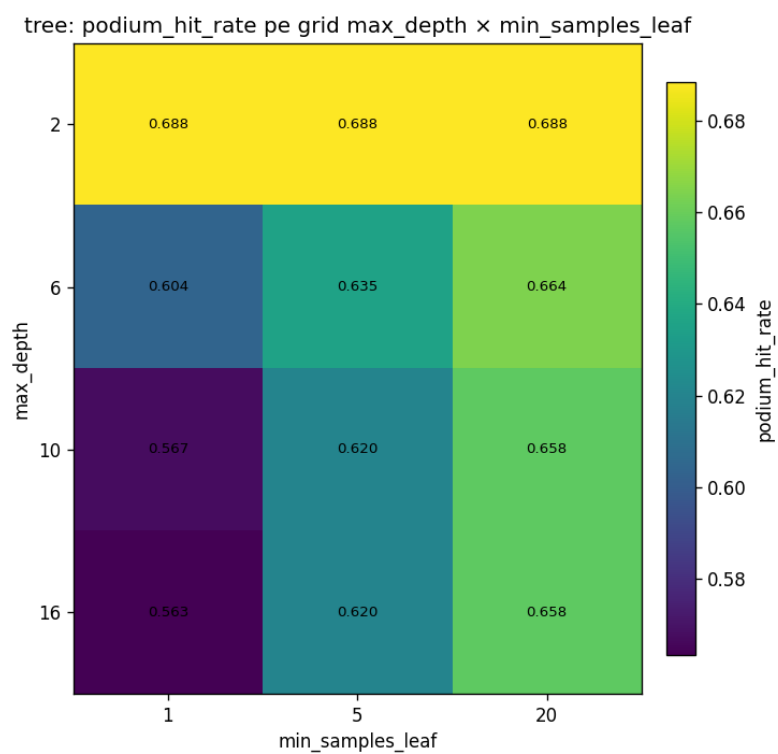


Figura 4.4 Căutare pe grilă 2D pentru arborele de decizie: rata de nimerire a podiumului în funcție de adâncimea maximă și de numărul minim de exemple pe frunză.

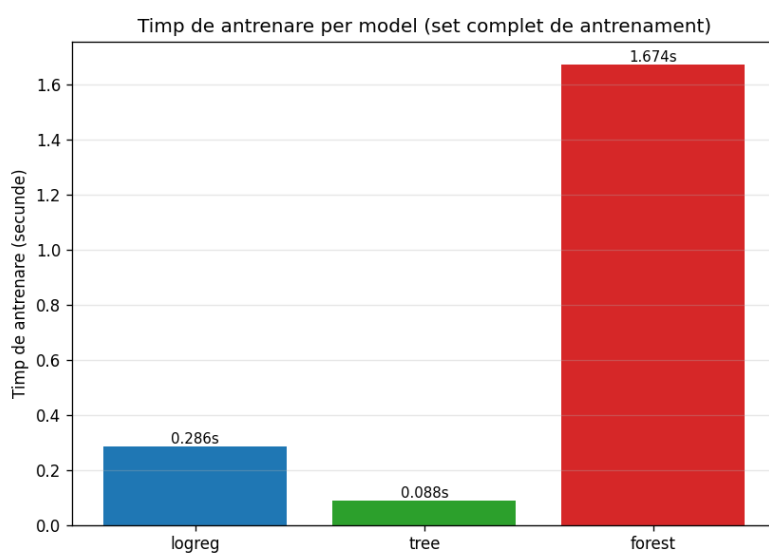


Figura 4.5 Timpul de antrenare al celor trei modele pe setul complet de antrenament.

4.6 Diagnostic de antrenare

Figura 4.6 prezintă două diagnostice de antrenare. Panoul (a) arată evoluția funcției de cost a regresiei logistice pe iterații, confirmând convergența coborârii pe gradient, în timp ce panoul (b) ilustrează scăderea erorii out-of-bag a pădurii aleatoare odată cu creșterea numărului de arbori, până la un platou în jurul valorii de 0,10. Complementar, Figura 4.7 prezintă distribuția ratei de nimerire a podiumului pe sezoanele de test: pădurea aleatoare are mediana cea mai bună, dar și o variație mai mare de la un sezon la altul, în timp ce regresia logistică este mai stabilă.

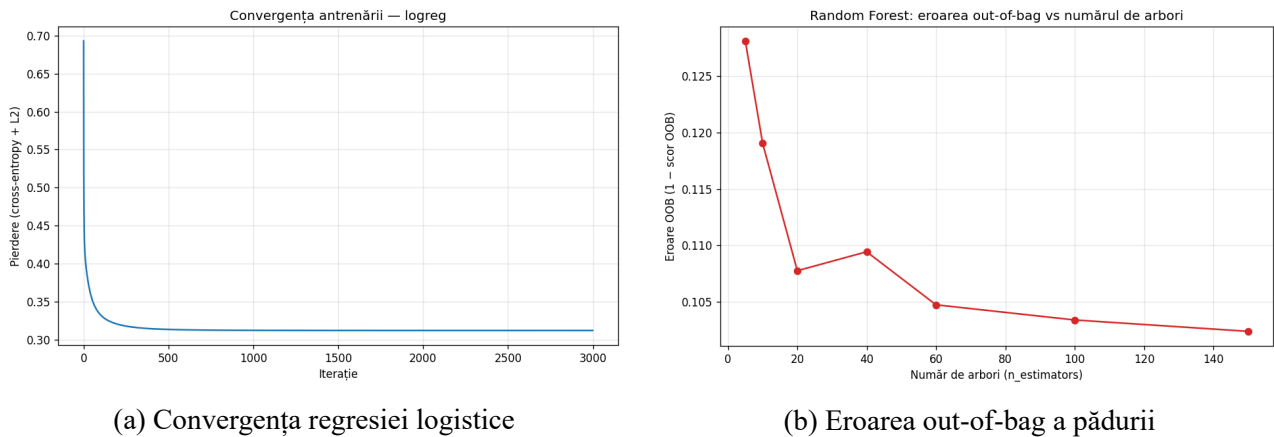


Figura 4.6 Diagnostic de antrenare: convergența (a) și eroarea out-of-bag (b).

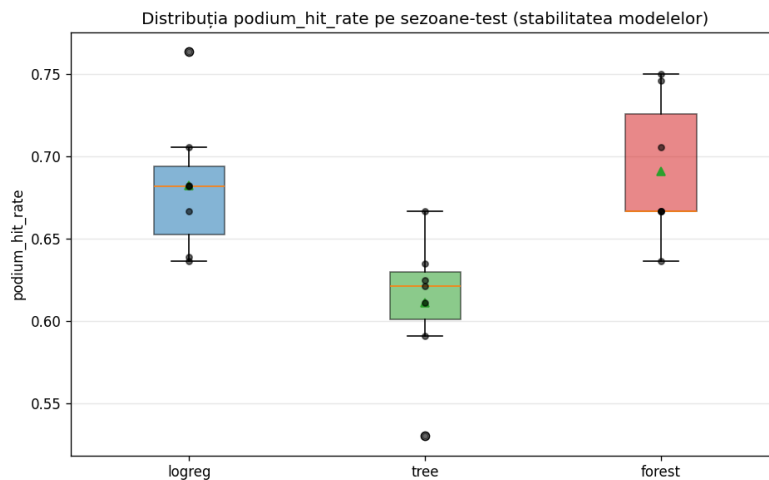


Figura 4.7 Distribuția ratei de nimerire a podiumului pe sezoanele de test (stabilitatea modelelor).

4.7 Sensibilitatea la pragul de decizie

Întrucât probabilitatea este transformată în etichetă printr-un prag, Figura 4.8 analizează variația metricilor de clasificare în funcție de acest prag pentru pădurea aleatoare. Se observă compromisul clasic precizie–recall: pe măsură ce pragul crește, precizia crește iar recall-ul scade, scorul F_1 atingând un maxim sub valoarea implicită de 0,5. Acest rezultat este în concordanță cu natura dezechilibrată a problemei.

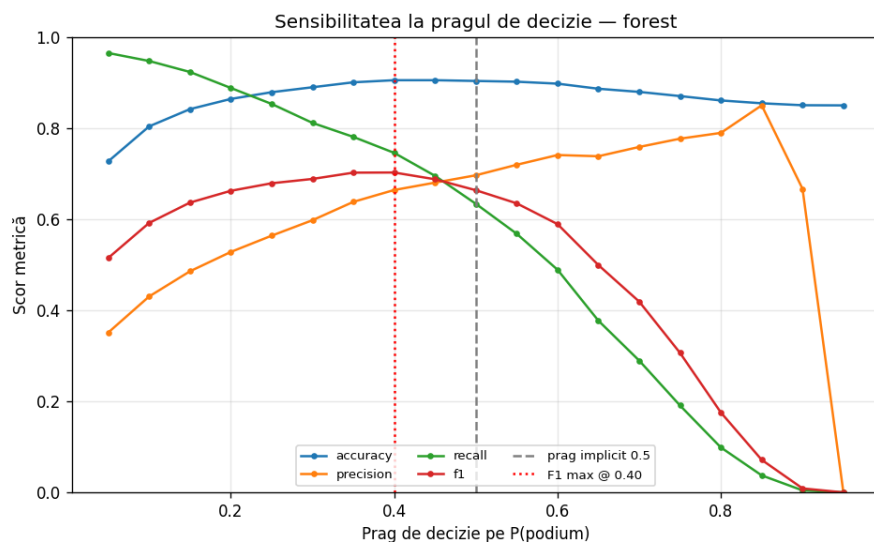


Figura 4.8 Sensibilitatea metricilor de clasificare la pragul de decizie, pentru pădurea aleatoare.

4.8 Discuție: acuratețea în prezența dezechilibrului

Un rezultat aparent paradoxal este faptul că acuratețea depășește 0,87 la toate modelele (Tabelul 4.1), în timp ce rata de nimerire a podiumului este de circa 0,69. Explicația rezidă în dezechilibrul de clase: deoarece doar aproximativ 15% dintre exemple sunt pozitive, un clasificator care prezice mereu “nu este podium” atinge deja o acuratețe de circa 85%. Prin urmare, acuratețea de 0,90 reprezintă o îmbunătățire modestă față de acest reper trivial, motiv pentru care lucrarea raportează ca metrici principale scorul F_1 , aria sub curba Precision-Recall și rata de nimerire a podiumului, în acord cu literatura [3], [4].

4.9 Studiu de caz: sezonul 2025

Pentru sezonul 2025, pădurea aleatoare prezice corect toți cei trei piloți din primele trei poziții ale clasamentului de campionat. Figura 4.9 compară punctele reale cu cele estimate, ilustrând capacitatea modelului de a reconstitui ierarhia campionatului din predicții agregate la nivel de cursă.

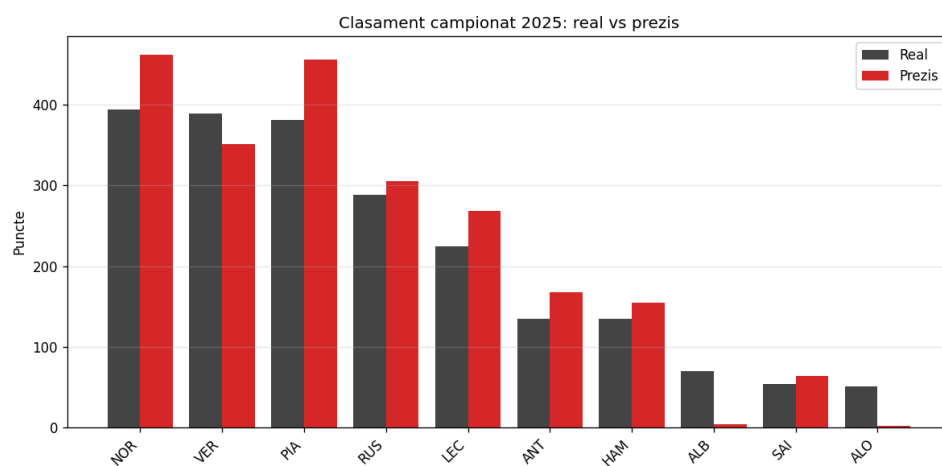


Figura 4.9 Clasamentul de campionat 2025: puncte reale față de cele estimate de pădurea aleatoare.

Capitolul 5

Concluzii și dezvoltări ulterioare

5.1 Concluzii

Lucrarea a atins obiectivele propuse. Cei trei algoritmi de învățare automată — regresia logistică, arborele de decizie și pădurea aleatoare — au fost implementați integral, fără biblioteci de învățare automată, și aplicați pe un set de date real din Formula 1, cu o etapă de inginerie a caracteristicilor strict cauzală care elimină scurgerea de informație. Studiul comparativ, realizat în condiții identice de antrenare și evaluare temporală, a evidențiat că:

- pădurea aleatoare oferă cea mai bună acuratețe și rată de nimerire a podiumului, dar la cel mai mare cost de calcul;
- regresia logistică atinge o capacitate de discriminare aproape identică (AUC și AP comparabile sau superioare) la o fracțiune din timpul de antrenare, fiind cea mai eficientă alegere în raport cu costul;
- arborele de decizie individual este în mod constant inferior, ceea ce confirmă avantajul regularizării și al metodelor de ansamblu;
- în prezența dezechilibrului de clase, acuratețea este o metrică înșelătoare, iar evaluarea trebuie să se bazeze pe F_1 , AP și metrici specifice problemei.

Dincolo de rezultatele numerice, valoarea principală a lucrării constă în metodologia riguroasă de evaluare (validare temporală fără scurgere de informație) și în instrumentația de analiză a hiperparametrilor și a costului de calcul.

5.2 Dezvoltări ulterioare

Pe termen scurt, soluția poate fi extinsă în mai multe direcții. În primul rând, includerea unor algoritmi suplimentari (de exemplu, mașini cu vectori suport sau metode de tip *gradient boosting*) ar îmbogăți studiul comparativ. În al doilea rând, calibrarea explicită a probabilităților ar putea îmbunătăți interpretarea acestora și agregarea în campionat. În al treilea rând, ingineria caracteristicilor poate fi rafinată prin includerea unor informații de cronometraj mai fine (ritmul în antrenamente, degradarea anvelopelor). Nu în ultimul rând, optimizarea automată a hiperparametrilor și paralelizarea antrenării pădurii ar reduce costul de calcul evidențiat în Capitolul 4.

Bibliografie

- [1] C. R. Harris et al., „Array Programming with NumPy,” *Nature*, vol. 585, nr. 7825, pp. 357–362, 2020. DOI: 10.1038/s41586-020-2649-2
- [2] F. Pedregosa et al., „Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] H. He și E. A. Garcia, „Learning from Imbalanced Data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, nr. 9, pp. 1263–1284, 2009. DOI: 10.1109/TKDE.2008.239
- [4] T. Saito și M. Rehmsmeier, „The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLoS ONE*, vol. 10, nr. 3, e0118432, 2015. DOI: 10.1371/journal.pone.0118432
- [5] C. M. Bishop, *Pattern Recognition and Machine Learning*. New York: Springer, 2006.
- [6] T. Hastie, R. Tibshirani și J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, a 2-a ed. New York: Springer, 2009.
- [7] L. Breiman, J. H. Friedman, R. A. Olshen și C. J. Stone, *Classification and Regression Trees*. Monterey, CA: Wadsworth și Brooks/Cole, 1984.
- [8] L. Breiman, „Random Forests,” *Machine Learning*, vol. 45, nr. 1, pp. 5–32, 2001. DOI: 10.1023/A:1010933404324
- [9] G. James, D. Witten, T. Hastie și R. Tibshirani, *An Introduction to Statistical Learning with Applications in R*. New York: Springer, 2013.
- [10] R. Kohavi, „A Study of Cross-Validation and Bootstrap for Accuracy Estimation and Model Selection,” în *Proceedings of the 14th International Joint Conference on Artificial Intelligence (IJCAI)*, 1995, pp. 1137–1143.
- [11] T. Fawcett, „An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, nr. 8, pp. 861–874, 2006. DOI: 10.1016/j.patrec.2005.10.010
- [12] A. Niculescu-Mizil și R. Caruana, „Predicting Good Probabilities with Supervised Learning,” în *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632. DOI: 10.1145/1102351.1102430
- [13] R. P. Bunker și F. Thabtah, „A Machine Learning Framework for Sport Result Prediction,” *Applied Computing and Informatics*, vol. 15, nr. 1, pp. 27–33, 2019. DOI: 10.1016/j.aci.2017.09.005

- [14] W. McKinney, „Data Structures for Statistical Computing in Python,” în *Proceedings of the 9th Python in Science Conference*, 2010, pp. 56–61. DOI: 10.25080/Majora-92bf1922-00a
- [15] J. D. Hunter, „Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, vol. 9, nr. 3, pp. 90–95, 2007. DOI: 10.1109/MCSE.2007.55
- [16] P. Schaefer și FastF1 contributors, *FastF1: Access to Formula 1 Timing Data and Telemetry*, <https://github.com/the0ehrly/Fast-F1>, Accesat: 2026-06-05, 2024.